

4-docker

Hector V.

14.10.2026

Ejercicio 1: Captura de pantalla de la creación de la red y del ping exitoso hacia el nombre del contenedor.



Figure 1: foto

```
  GNU nano 9.2                    /srv/docker/dck1/compose.yml
services:
  mi_cache:
    image: redis:alpine
    container_name: mi_cache
    networks:
      - red_practica

  alpine:
    image: alpine
    container_name: prueba_dns
    command: sh -c "ping -c 4 mi_cache"
    networks:
      - red_practica

networks:
  red_practica:
    driver: bridge
```

Figure 2: foto

```
~ 1m 32s
❯ docker compose up -d mi_cache
  docker compose run --rm alpine
```

Figure 3: foto

```
/srv/docker/dck1
❯ sudo docker compose up -d mi_cache
  sudo docker compose run --rm alpine
[+] up 12/12
 ✔ Image redis:alpine          Pulled
 ✔ Network dck1_red_practica Created
 ✔ Container mi_cache          Started
Image alpine Pulling
e2de96513ba9 Pulling fs layer 0B
797dd00a0fc7 Download complete 0B
6d0606d1815c Download complete 0B
e2de96513ba9 Downloading 1.049MB
e2de96513ba9 Downloading 1.049MB
e2de96513ba9 Downloading 1.049MB
e2de96513ba9 Downloading 1.049MB
e2de96513ba9 Downloading 1.049MB
e2de96513ba9 Downloading 1.049MB
e2de96513ba9 Downloading 1.049MB
e2de96513ba9 Downloading 2.097MB
e2de96513ba9 Downloading 2.097MB
e2de96513ba9 Downloading 2.097MB
e2de96513ba9 Downloading 2.097MB
e2de96513ba9 Downloading 2.097MB
e2de96513ba9 Downloading 2.097MB
e2de96513ba9 Downloading 3.146MB
e2de96513ba9 Downloading 3.146MB
e2de96513ba9 Downloading 3.146MB
e2de96513ba9 Downloading 3.146MB
e2de96513ba9 Downloading 3.146MB
e2de96513ba9 Download complete 0B
e2de96513ba9 Extracting 1B
e2de96513ba9 Pull complete 0B
Image alpine Pulled
Container dck1-alpine-run-e464f3dddeb7 Creating
Container dck1-alpine-run-e464f3dddeb7 Created
PING mi_cache (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.070 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.061 ms
64 bytes from 172.18.0.2: seq=2 ttl=64 time=0.188 ms
64 bytes from 172.18.0.2: seq=3 ttl=64 time=0.107 ms
```

Figure 4: foto

Ejercicio 2: Captura del bind mount funcionando en el navegador e investigación teórica sobre volúmenes.

```
 GNU nano 9.2                          compose.yml
services:
  nginx:
    image: nginx:alpine
    container_name: web_nginx
    ports:
      - "8080:80"
    volumes:
      - ./web_dev:/usr/share/nginx/html

  prueba_volumen:
    image: alpine
    command: sh -c "echo 'segasegaseganega' > /datos/prueba.txt && cat /datos/prueba.txt"
    volumes:
      - datos_persistentes:/datos

volumes:
  datos_persistentes:
```

Figure 5: foto

```
/srv/docker/dck2
❯ sudo docker compose up -d nginx
  sudo docker compose run --rm prueba_volumen
[+] up 1/1
 ✔ Container web_nginx Running
Container dck2-prueba_volumen-run-d45f9cf5c22b Creating
Container dck2-prueba_volumen-run-d45f9cf5c22b Created
segasegaseganega
```

Figure 6: foto

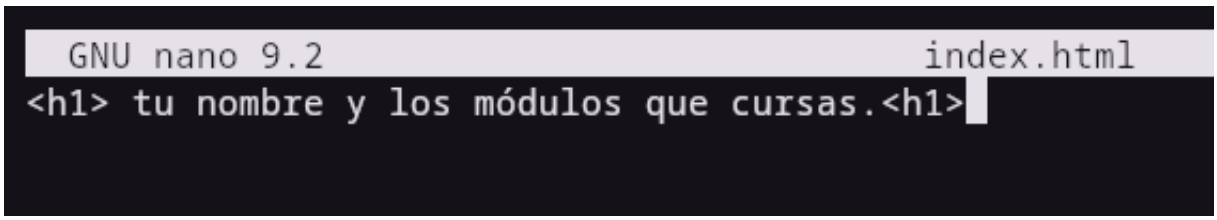


Figure 7: foto

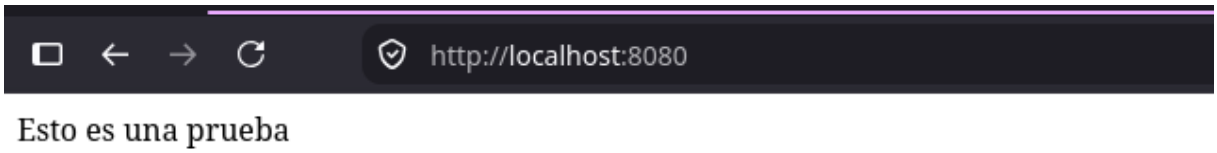


Figure 8: foto

Explicacion: El blind mount es util para debuggear y hacer funcionar el servicio, ademas de configurarlo o repararlo de una manera mas profunda, mientras que los volumenes son mas recomendables para produccion.

Ejercicio 3: Código del Dockerfile, comando docker build y captura de la web personalizada en ejecución.

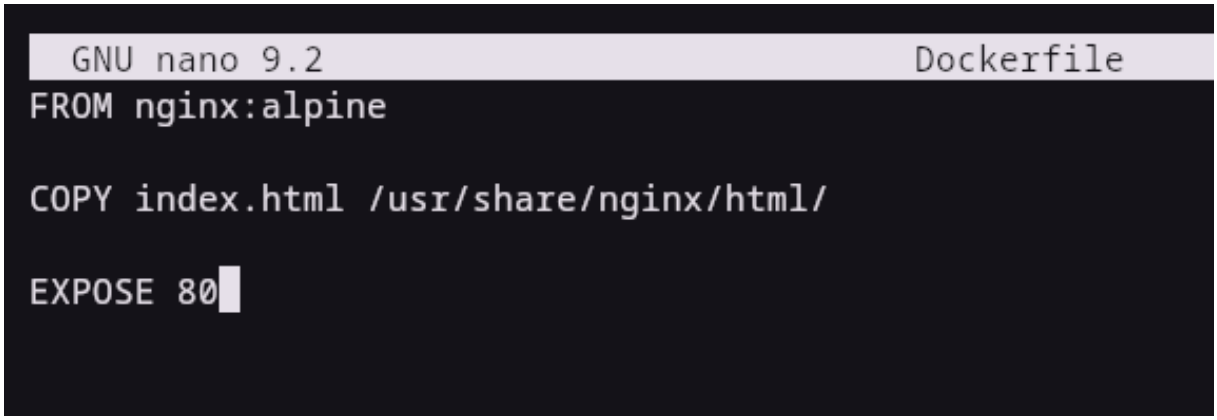


Figure 9: foto

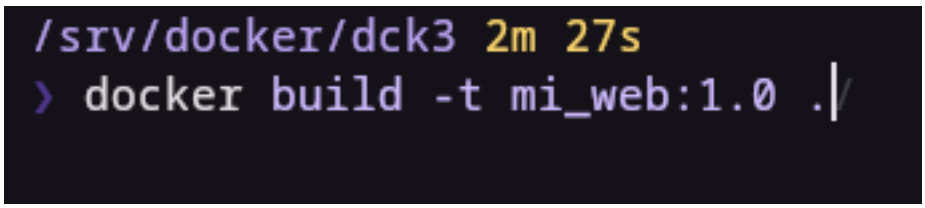


Figure 10: foto



Figure 11: foto

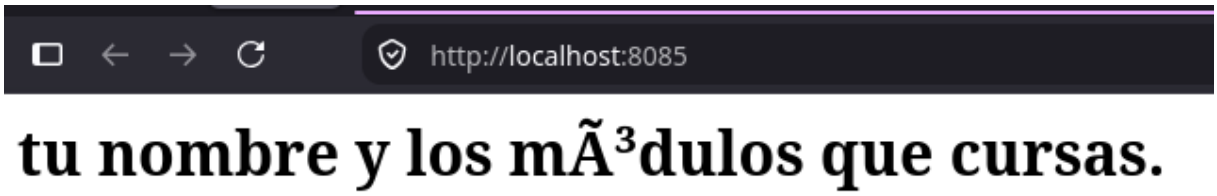


Figure 12: foto

Ejercicio 4: Código completo de docker-compose.yml, capturas de los comandos docker compose up -d, docker compose ps y verificación en el navegador.

```
/srv/docker/dck4
› cat compose.yml
services:
  app:
    build: .
    container_name: mi_web
    ports:
      - "8080:80"
    volumes:
      - ./web:/usr/share/nginx/html
    networks:
      - red_interna

  cache:
    image: redis:alpine
    container_name: mi_cache
    volumes:
      - datos_cache:/data
    networks:
      - red_interna

networks:
  red_interna:
    driver: bridge

volumes:
  datos_cache:
```


Figure 13: foto

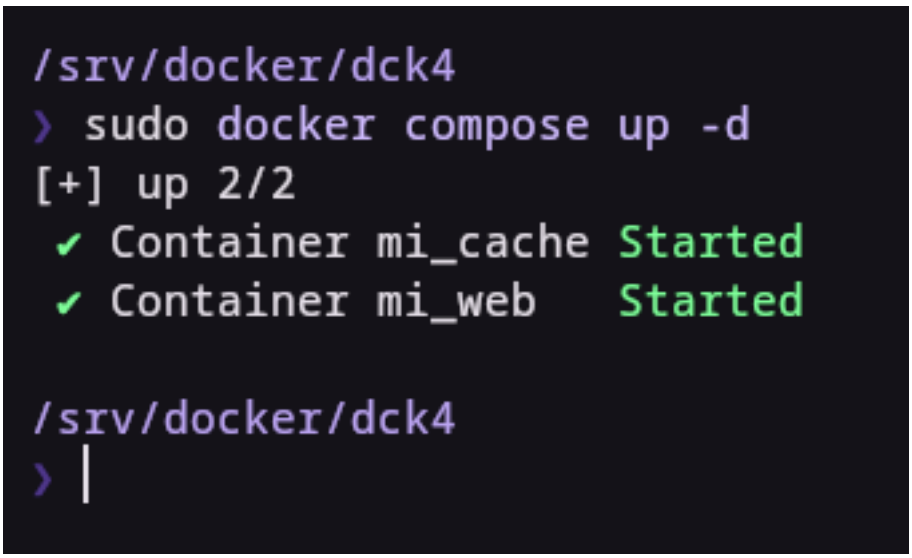


Figure 14: foto



Figure 15: foto